HW 7 coming soon.

Quiz 9 is due next Monday.

https://www.quantamagazine.org/amateur-mathematicians-find-fifth-busy-beaver-turing-machine-20240702/

Class 19: Halting Problem

University of Virginia
CS3120: DMT2
https://weikailin.github.io/cs3120-toc
Wei-Kai Lin

Plan

Computability

Computable functions

An uncomputable function

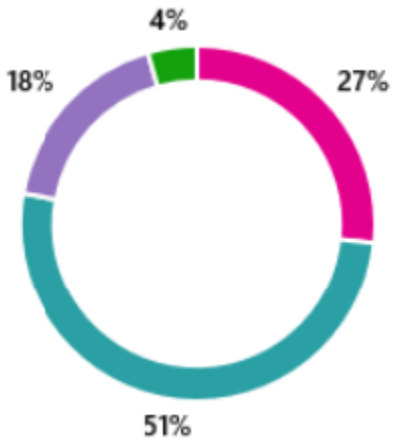
Textbook [TCS] Section 9

https://introtcs.org/public/lec_08_uncomputability.html
Universality and uncomputability

Mid-semester survey (Anonymous)

The average number of hours (per week) that I spent outside of class preparing for CS 3120

Less than 1	0
1-3	12
4-6	23
7-9	8
10 or more	2



Course Materials

I had access to the course materials I needed to learn, including access to required textbooks, course readings, technology tools, and other materials required for class assignments and projects.

What would you like the instructor to know?

Resources and materials:

“one of the chapter sections from a different book” (Sipser, to prove the equivalence of Reg Exp and DFA)

“sometimes get confused on proofs from the textbook”

Partly bcs it is still evolving, partly bcs time constraints.

Eg, Barak’s textbook is open source, we can improve it.

I want my slides and assignments to be consistent (locally). Let me know if it is not.

Material and Delivery (Difficulty)

Definitions: computation models are necessary; others are mostly shorthand.

A confusion may arise when forgetting a defn:

Eg, “x = x + 1” in programming 101,

“language” in this course,
a Turing Machine M(x) does not halt is denoted by $\perp$.

Grades and Midterms

“short midterms…” “midterms being 20 minutes… an inaccurate assessment”

Let’s say 4 T/F/Choice and 1 short answer next Midterm.

Fewer HW questions & lower weight: TA workloads, GenAI.

General suggestion: “optional” questions are necessary.

Solving problems?

In this course, we ask questions to clarify computation.

Non-goals: writing reg exp, DFA, or TM programs.

Writing proofs is important, but also a methodology.

Some exercised many times: proof by contradiction, diagonalization, counting argument, induction, reduction (called problem solving)

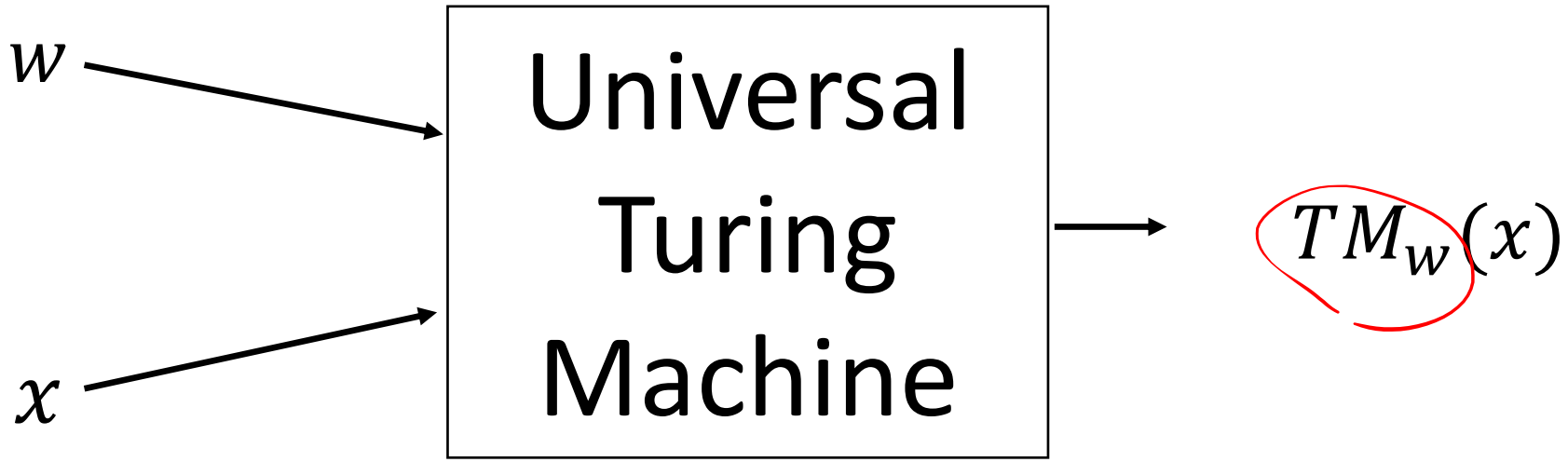
Recap

Recap: Universal Turing Machine

For any $w \in \{0,1\}^*$, let TM_w be the Turing machine M

such that w represents M.

If w does not represent any TM, let TM_w be the TM that output 0 and halt (ignore input).



Recap: Computable Functions

Definition (Definition 7.2):

For any Boolean function $F: \{0,1\}^* \to \{0,1\}^*$ and any Turing Machine M, we say M ***computes*** F iff for all $x \in \{0,1\}^*$, $M(x) = F(x)$.

We say F is ***computable*** if and only if there exists a Turing machine M such that computes F.

Big Idea 9

We can precisely define what it means for a function to be computable by any possible algorithm.

When the function F is Boolean (output 1 bit), we sometimes say F is **decidable,** which stresses both computable and Boolean output.

When a TM M computes on a input x and outputs **1**, ie M(x) = 1, we say M **accepts** x.

When a TM M computes on a input x and outputs **0**, ie M(x) = 0, we say M **rejects** x.

Examples of Computable Functions

- ADD(x,y), MUL(x,y), EXP(x,y), …
- Log(x,i), Cos(x,i), … for precision ~~I~~ i SQRT(x, i)
- SORT($x_1, \ldots, x_n$), … Count(x_1 … x_n)
- EVEN(x), PRIME(x), …
- More?

Some Numbers are Uncomputable!

$$|\,TMs\,| \;=\; |\,\text{finite binary strings}\,| \;=\; |\,\mathbb{N}\,|$$

$$|\,\mathbb{R}\,| = |\,pow(\mathbb{N})| > |\,\mathbb{N}\,|$$

Is there an *interesting function* that cannot be computed? (Uncomputability)

NSA Function (Not Self-Accept)

A Turing Machine, $M = (C, \Sigma, \delta)$, **accepts** a string, x, if $M(x) = 1$.

$$NSA(x) = \begin{cases} 0, & \text{if } TM_x(x) = 1 \\ 1, & \text{otherwise} \end{cases}$$

Is NSA computable?

Computing NSA?

A Turing Machine, $M = (C, \Sigma, \delta)$, **accepts** a string, x, if $M(x) = 1$.

$$NSA(x) = \begin{cases} 0, & \text{if } TM_x(x) = 1 \\ 1, & \text{otherwise} \end{cases}$$

Universal Turing Machine

$w, x \longrightarrow \boxed{\mathcal{U}} \longrightarrow$ Result of running TM_w on x

Build a TM:

$$M(x) = \textbf{if } (\mathcal{U}(x, x) = 1) \text{ output } 0$$
$$\textbf{else} \text{ output } 1$$

Computing NSA?

A Turing Machine, $M = (C, \Sigma, \delta)$, **accepts** a string, x, if $M(x) = 1$.

$$NSA(x) = \begin{cases} 0, & \text{if } TM_x(x) = 1 \\ 1, & \text{otherwise} \end{cases}$$

M **does not compute** *NSA*, since it may not halt:
if $\mathcal{U}(x, x)$ continues infinitely, M does not output correctly!

Build a TM:

$$M(x) = \textbf{if } (\mathcal{U}(x, x) = 1) \text{ output } 0$$
$$\textbf{else} \text{ output } 1$$

Note: this does not **prove** that NSA is uncomputable, since it doesn't show there isn't some other way to compute it (eg, without using $\mathcal{U}(w, x)$)

NSA is Uncomputable

How to reach a contradiction?

Proof by contradiction:

Assume some TM, $M_{\cancel{x}}(x)$, computes *NSA*.

Table of Machines

Input, x

TM, w	ε	0	1	00	01	10	11	000	001	010	…
ε											…
0											…
1	✓	✓		✓				✓			
00											
01	✓		✓				✓				
10				✓							
11											
000	✓ $TM_{000}(\text{""})=1$	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
…											

✓ in (w, x) means $TM_w(x) = 1$

Not the actual table (of course!)

What's this table?

Table of Machines

Input, x

TM, w

NSA	ε	0	1	00	01	10	11	000	001	010	…
ε	✓										…
0		✓									…
1	✓	✓	✓	✓				✓			
00				✓							
01	✓		✓		✓		✓				
10				✓		✓					
11							✓				
000	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
…									✓		

✓ in (w, x) means $TM_w(x) = 1$

What's the diagonal? $TM_w(w) \stackrel{?}{=} 1$

What's the "negation of the diagonal"? $NSA(w)$

NSA is Uncomputable

Definition:

A Boolean function $F: \{0,1\}^* \to \{0,1\}$ is ***computable*** if and only if there exists a Turing machine M such that for all $x \in \{0,1\}^*$, $M(x) = F(x)$.

Proof by contradiction:

Assume some TM, $M(x)$, computes *NSA(x)*.

There must be some string, w such that, $M = TM_w$.

Let's consider the outcome of

$$\mathcal{U}(w,w) = TM_w(w) = M(w)$$

halt all w

halt for all w

What's $\mathcal{U}(w,w)$?

What's $NSA(w)$?

Does $\mathcal{U}(w,w)$ halt?

NSA is Uncomputable

$$NSA(x) = \begin{cases} 0, & \text{if } TM_x(x) = 1 \\ 1, & \text{otherwise} \end{cases}$$

Proof by contradiction:

Assume some TM, $M(x)$, computes *NSA(x)*.

There must be some string, w such that, $M = TM_w$.

Let's consider the outcome of $\mathcal{U}(w, w)$.

We have $= TM_w(w) = M(w) = NSA(w)$

$\shortparallel$

0

What's $NSA(w)$?

Fix w,

Option 1: $NSA(w) = 0$.

$TM_w(w) = 0 \Rightarrow NSA(w) = 1$

NSA is Uncomputable (Section 9.2)

$$NSA(x) = \begin{cases} 0, & \text{if } TM_x(x) = 1 \\ 1, & \text{otherwise} \end{cases}$$

Proof by contradiction:

Assume some TM, $M(x)$, computes *NSA(x)*.

There must be some string, w such that, $M = TM_w$.

Let's consider the outcome of $\mathcal{U}(w, w)$.

We have $= TM_w(w) = M(w) = NSA(w)$ (eq)

for some fixed w

Option 1: $NSA(w) = 0$.

By def of NSA: $TM_w(w) = 1$

By (eq): $TM_w(w) = NSA(w) = 0$

Option 2: $NSA(w) = 1$.

By def of NSA: $TM_w(w) \neq 1$ $\Rightarrow NSA(w)=0$

By (eq): $TM_w(w) = NSA(w) = 1$

Is *NSA Interesting*? What does it mean?

$$NSA(x) = \begin{cases} 0, & \text{if } TM_x(x) = 1 \\ 1, & \text{otherwise} \end{cases}$$

Proof by contradiction:

Assume some TM, $M(x)$, computes *NSA(x)*.

There must be some string, w such that, $M = TM_w$.

Let's consider the outcome of $\mathcal{U}(w, w)$.

We have $= TM_w(w) = M(w) = NSA(w)$ (eq)

Option 1: $NSA(w) = 0$.

By def of NSA: $TM_w(w) = 1$

By (eq): $TM_w(w) = NSA(w) = 0$

Option 2: $NSA(w) = 1$.

By def of NSA: $TM_w(w) \neq 1$

By (eq): $TM_w(w) = NSA(w) = 1$

Contradiction: M_A must not exist! Therefore, *NFA* is uncomputable.

Halting Problem

Halting Problem (Section 9.3)

$$HALT(w, x) = \begin{cases} 1, & \text{if } TM_w \text{ terminates on } x \\ 0, & \text{otherwise} \end{cases}$$

Can we show *HALT* is computable?

~~**Strategy:** look for an infinite loop in w~~
~~**Strategy:** run $TM_w(x)$ and look for repeated states~~

Exercise: write a TM that increments a counter infinitely

Prove Halting Problem

$$NSA(x) = \begin{cases} 0, & \text{if } TM_x(x) = 1 \\ 1, & \text{otherwise} \end{cases}$$

$$HALT(w, x) = \begin{cases} 1, & \text{if } TM_w \text{ terminates on } x \\ 0, & \text{otherwise} \end{cases}$$

How can we show *HALT* is uncomputable?

Strategy 1: show we can use a machine that computes it to produce a contradiction (like we did to show NSA is uncomputable)

Prove Halting Problem

$$NSA(x) = \begin{cases} 0, & \text{if } TM_x(x) = 1 \\ 1, & \text{otherwise} \end{cases}$$

$$HALT(w, x) = \begin{cases} 1, & \text{if } TM_w \text{ terminates on } x \\ 0, & \text{otherwise} \end{cases}$$

How can we show *HALT* is uncomputable?

Strategy 1: show we can use a machine that computes it to produce a contradiction (like we did to show NSA is uncomputable)

Strategy 2: show we can use a machine that computes it to produce a machine that computes NSA.

Prove by Reduction

$$NSA(x) = \begin{cases} 0, & \text{if } TM_x(x) = 1 \\ 1, & \text{otherwise} \end{cases}$$

$$HALT(w, x) = \begin{cases} 1, & \text{if } TM_w \text{ terminates on } x \\ 0, & \text{otherwise} \end{cases}$$

If there exists M_B that solves B, then we have M_A that solves A

Strategy 2: show we can use a machine that computes it to produce a machine that computes NSA.

$$NSA(x) = \begin{cases} 0, & \text{if } TM_x(x) = 1 \\ 1, & \text{otherwise} \end{cases}$$

$$HALT(w, x) = \begin{cases} 1, & \text{if } TM_w \text{ terminates on } x \\ 0, & \text{otherwise} \end{cases}$$

Proving *HALT* is Uncomputable

Assume $HALT$ is computable.

By the assumption (and definition of computable), there exists some TM M_{HALT} that computes $HALT$.

We can use M_{HALT} to build a machine that decides NSA:

$$NSA(x) = \begin{cases} 0, & \text{if } TM_x(x) = 1 \\ 1, & \text{otherwise} \end{cases}$$

$$HALT(w, x) = \begin{cases} 1, & \text{if } TM_w \text{ terminates on } x \\ 0, & \text{otherwise} \end{cases}$$

Proving *HALT* is Uncomputable

Assume $HALT$ is computable.

By the assumption (and definition of computable), there exists some TM M_{HALT} that computes $HALT$.

We can use M_{HALT} to build a machine that decides NSA:

$M_{NSA}(x)$: 1. $h = M_{HALT}(x, x)$
2. if $h = 1$: return NOT($\mathcal{U}(x, x) = 1$)
else: 1

$$NSA(x) = \begin{cases} 0, & \text{if } TM_x(x) = 1 \\ 1, & \text{otherwise} \end{cases}$$

$$HALT(w, x) = \begin{cases} 1, & \text{if } TM_w \text{ terminates on } x \\ 0, & \text{otherwise} \end{cases}$$

Proving *HALT* is Uncomputable

Assume $HALT$ is computable.

By the assumption (and definition of computable), there exists some TM M_{HALT} that computes $HALT$.

We can use M_{HALT} to build a machine that decides NSA:

$M_{NSA}(x)$: 1. $h = M_{HALT}(x, x)$
2. if $h = 1$: return NOT($\mathcal{U}(x, x) = 1$)
else: 1

Thus, since we know M_{NSA} does not exist, but if we had M_{HALT} we could build it, we have a contradiction! This proves that M_{HALT} must not exist which means $HALT$ is not computable.

$$HALT(w,x) = \begin{cases} 1, & \text{if } TM_w \text{ terminates on } x \\ 0, & \text{otherwise} \end{cases}$$

Theorem: *HALT* is Uncomputable

Implications?

Can we say a given TM halts on all inputs?

Can we compute HALT(w,x) for all 100-bit w and x?

Logistics

HW 7 coming later this week.

Quiz 9 due next Monday.

Plan

Computability

Rice's Theorem

Next Module: Complexity (P, NP, …)

Textbook [TCS] Section 9

https://introtcs.org/public/lec_08_uncomputability.html
Universality and uncomputability